

Digital, Analog & PWM Signals

Welcome to Lesson 08! This is where things get really interesting — we're diving into the three types of signals that every robotics beginner *must* understand. Digital signals (simple ON/OFF), analog signals (smooth and continuous), and PWM (the clever trick that lets digital pins fake analog output). By the end of these notes, you'll be reading sensors, controlling LEDs, and understanding exactly what's happening at the pin level. Let's get into it! 🖍️

★ Digital Signal — The Basics

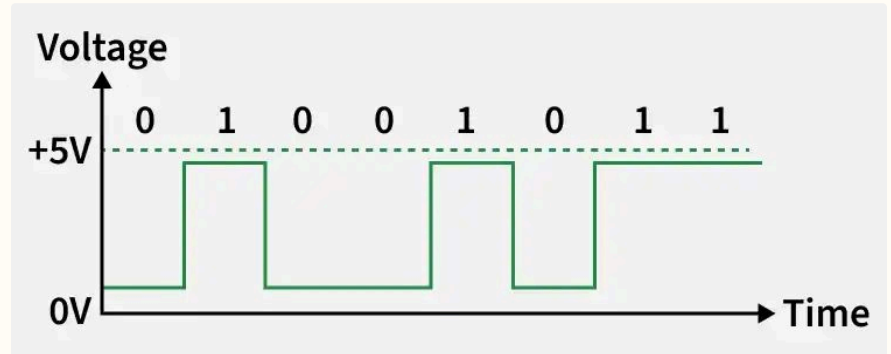
A **digital signal** is the simplest type of electrical signal. It has only **two possible states** — no in-between, no gray area:

- **HIGH** = 1 = ON = ~3.3V or 5V (depends on your board)
- **LOW** = 0 = OFF = 0V (GND)

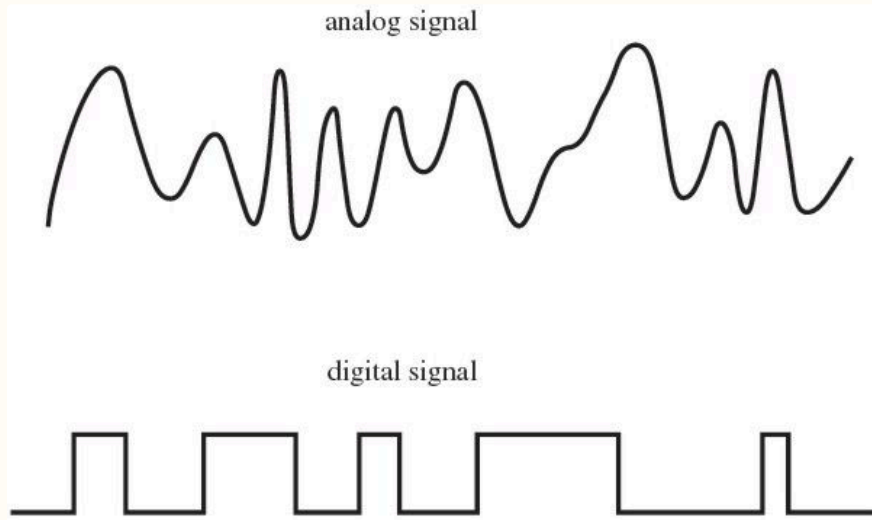
Think of it like a **light switch** — either fully ON or fully OFF. There's no "half-on" in the digital world. Your microcontroller's GPIO pins send and read these signals constantly.



Digital I/O: A pin configured as **output** sends HIGH/LOW. A pin as **input** reads whether a signal is HIGH or LOW. Super useful for buttons, LEDs, relays!



★ Analog Signal — Continuous World



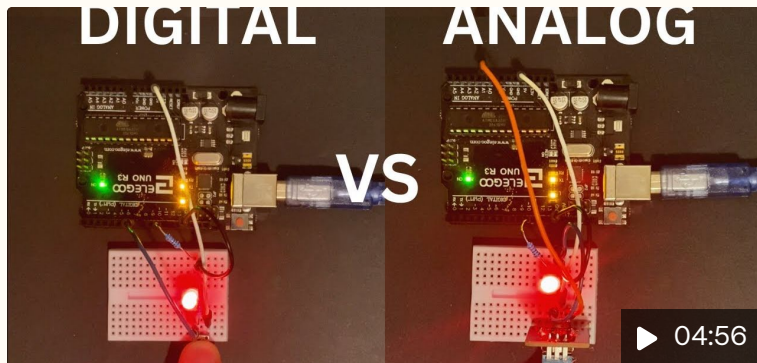
An **analog signal** is **continuous** — it can take *any* value within a range. Unlike digital's two states, analog smoothly varies over time, just like the real world does!

Real-world examples:

- 🌡️ Temperature sensor output (smoothly rises and falls)
- ☀️ LDR / Light sensor (changes with light intensity)
- 📏 Potentiometer knob position (0 to full rotation)
- 🎤 Microphone audio signal (constantly changing)

Most things in the physical world are analog. Our microcontrollers are digital — so we need a way to **bridge** the two worlds. That's where **ADC** comes in (more on that soon!).

★ Digital vs Analog — Side by Side



YouTube



Arduino Basics: Digital And Analog For Input And Output

In today's tutorial we'll learn digital and analog inputs and outputs. We'll build a simple circuit with an LED, button, and potentiometer to demonstrate how to...

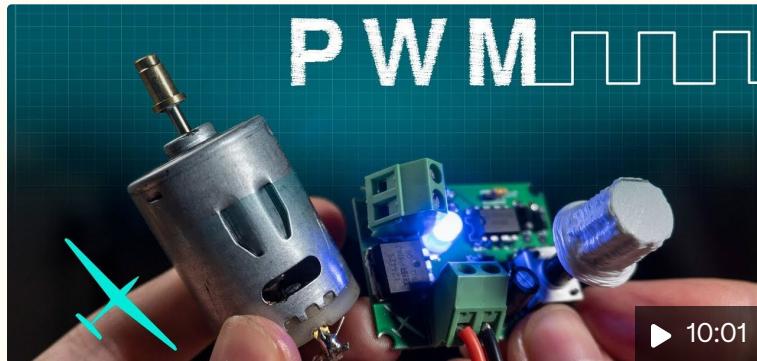
Here's a quick comparison to lock this in your memory. The key difference: digital is **discrete** (only two values), analog is **continuous** (infinite values in a range).

Feature	Digital Signal	Analog Signal
Values	Only 2 — HIGH (1) or LOW (0)	Continuous range — any value possible
Waveform	Square wave (sharp edges)	Smooth curve (sine, ramp, etc.)
Noise	Less affected by noise	More susceptible to noise
Examples	LED, button, relay, buzzer	Temperature, LDR, pot, mic
Processing	Direct — MCU understands natively	Needs ADC to be read by MCU
Real-world fit	Computers, logic circuits	Physical sensors, audio, light



★ **Remember:** The physical world is analog. Microcontrollers are digital. We use **ADC** to convert analog → digital, and **PWM** to fake analog output from digital pins!

★ PWM — Pulse Width Modulation



YouTube

How PWM works | Controlling a DC motor with a homemade circuit

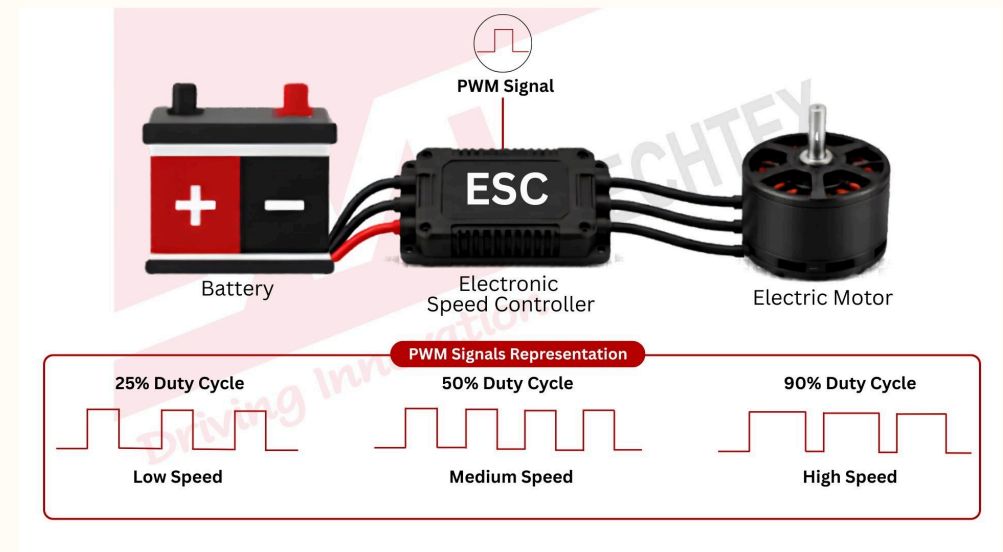
\$2 for 10pcs PCBs (24 hours turnaround time): <https://jlcpcb.com> They work during chinese new year! This is a very cool electronic project to control DC load...

PWM stands for **Pulse Width Modulation**. It's a clever technique where a *digital* pin rapidly switches between HIGH and LOW to **simulate** an analog output. The key idea: control *how long* the signal stays HIGH vs LOW in each cycle.

Key terms to know:

- **Ton** — Time the signal is HIGH (ON time)
- **Toff** — Time the signal is LOW (OFF time)
- **Period (T)** = Ton + Toff (one full cycle)
- **Frequency** = $1 / \text{Period}$

The faster the switching, the smoother the effect. On ESP32, PWM frequency is typically **1 kHz – 50 kHz**.



PWM

★ Duty Cycle — The Heart of PWM

Duty Cycle tells us what *percentage* of one period the signal is HIGH. It's the number that controls brightness, speed, power — everything!

$$\text{Duty Cycle} = \text{Ton} \div (\text{Ton} + \text{Toff}) \times 100\%$$

25% Duty Cycle

Signal is HIGH for $\frac{1}{4}$ of the period. LED appears **dim**. Motor runs **slow**.

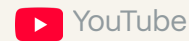
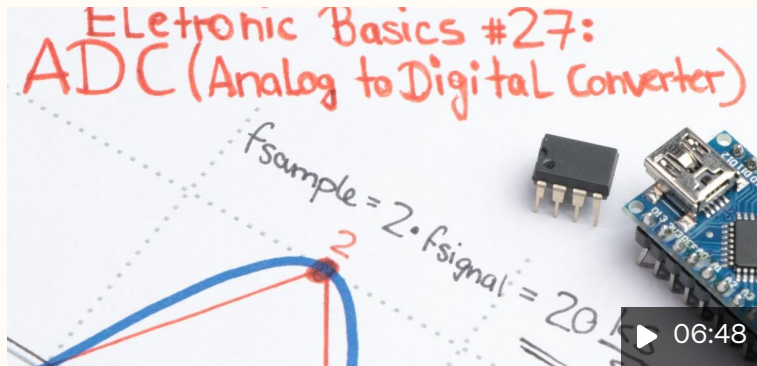
50% Duty Cycle

Signal is HIGH for $\frac{1}{2}$ of the period. LED at **half brightness**. Medium speed.

75% Duty Cycle

Signal is HIGH for $\frac{3}{4}$ of the period. LED appears **bright**. Motor runs **fast**.

★ ADC — Analog to Digital Converter



Electronic Basics #27: ADC (Analog to Digital Converter)

Previous video: <https://youtu.be/pDtqxf7BLiM> Electronic Basics #10: DAC: <https://youtu.be/Y2OPnrgb0pY> Facebook:...



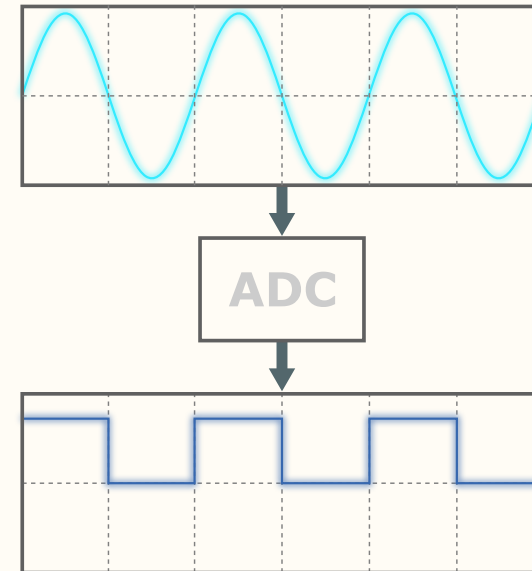
An **ADC (Analog-to-Digital Converter)** is a circuit inside your microcontroller that **converts a continuous analog voltage** into a **digital number** the CPU can process. Without it, your MCU literally cannot read sensors like LDRs, temperature sensors, or potentiometers!

Why do we need it? The real world is analog (smooth voltages). Your ESP32 is digital (only understands 1s and 0s). ADC bridges the gap! 🔗

Resolution Formula:

$$\text{Resolution} = V_{\text{ref}} \div 2^n$$

where **n** = number of bits



★ ADC Resolution — 10-bit vs 12-bit

Higher bit resolution = **more precise** readings. Let's break it down with the formula **Resolution = Vref / 2ⁿ** (assuming Vref = 3.3V):

ADC Type	Bits (n)	Levels (2 ⁿ)	Resolution (at 3.3V)
10-bit ADC	10	1024 levels	3.3V ÷ 1024 ≈ 3.22 mV
12-bit ADC	12	4096 levels	3.3V ÷ 4096 ≈ 0.81 mV

10-bit Example (Arduino)

Input: 1.65V → Output: **512**

Half of 1024 = half voltage. Simple!

12-bit Example (ESP32)

Input: 1.65V → Output: **2048**

Half of 4096 = half voltage. More precise!



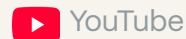
★ ESP32 has a **12-bit ADC** by default (0–4095). Arduino Uno has **10-bit ADC** (0–1023). More bits = finer detail in your sensor readings!

★ The 4 Essential Functions

ARDUINO 101



DIGITAL READ & WRITE ▶ 08:56



Arduino Lesson 2 – digitalRead & digitalWrite

The led's wiring can be found here, you can wire it up using a breadboard, although I have it soldered onto a circuit board because it's easier that way...

These are the **core functions** you'll use in almost every robotics project. Know them well! 🔥

1

digitalRead(pin)

Reads a digital pin. Returns **HIGH** (1) or **LOW** (0). Used for buttons, switches, digital sensors.

2

digitalWrite(pin, value)

Sets a digital pin to **HIGH** or **LOW**. Used for LEDs, relays, buzzers. Example: `digitalWrite(2, HIGH);`

3

analogRead(pin)

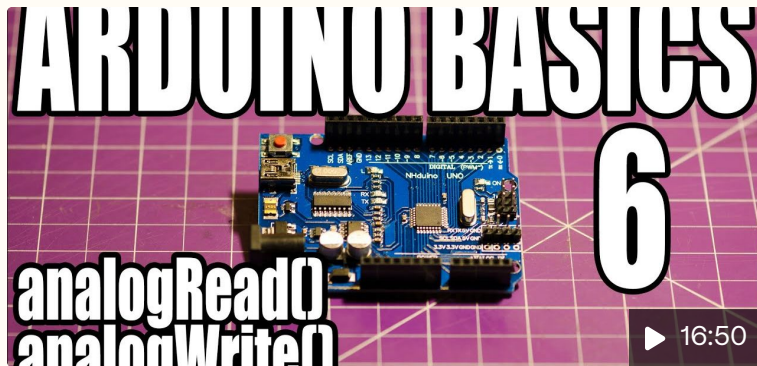
Reads an analog pin via ADC. Returns a number: **0–1023** (Arduino) or **0–4095** (ESP32). Used for sensors.

4

analogWrite(pin, value)

Outputs PWM on a pin. Value: **0–255** (0 = off, 255 = full). Used for LED brightness, motor speed control.

★ Project: LDR Controls LED Brightness



YouTube

Arduino Basics – #6 – analogRead() & analogWrite()

Video 6 in my arduino basics series. In this video, I'll guide you through some sketch instructions you'll want to familiarize yourself with: analogRead()...

Here's a **real practical example** that ties everything together — an **LDR (Light Dependent Resistor)** reading ambient light and using that value to control an **LED's brightness via PWM**. This is exactly how automatic night lights work!

How it works:

1. LDR output → `analogRead(A0)` → value 0–4095
2. Map the value: `map(val, 0, 4095, 0, 255)`
3. Send to LED: `analogWrite(D5, brightness)`
4. More light = brighter LED. Less light = dimmer LED.



Components: ESP32, LDR + 10kΩ resistor (voltage divider), LED + 220Ω resistor, breadboard, jumper wires.



Bright Room

LDR reads HIGH → PWM ~200 → LED bright



Dark Room

LDR reads LOW → PWM ~30 → LED dim



Key Concept

Analog sensor → ADC → PWM output.
Full circle!